Circular Queue Using Array
Program:

```
#include <stdio.h>
#define SIZE 5
int items[SIZE];
int front = -1, rear = -1;
int isFull()
{
 if ((front == rear + 1) || (front == 0 && rear == SIZE - 1))
   return 1;
return 0;
}

int isEmpty()
{
if (front == -1)
  return 1;
return 0;
}

void enQueue(int element)
{
if (isFull())
{
 printf("\nQueue is full!!\n");
}
else
{
if (front == -1) {
front = 0;
rear = 0;
} else {
rear = (rear + 1) % SIZE; }
}
items[rear] = element;
printf("\nInserted -> %d", element);
  }
}
int deQueue()
{
int element;
if (isEmpty())
```

```
{
printf("\nQueue is empty!!\n");
return -1;
} else {
element = items[front];
 if (front == rear)
 front = -1;
 rear = -1;
}
else {
front (front + 1) % SIZE;
}
printf("\nDeleted element -> %d\n", element);
return element;
   }
}

void display()
{
int i;
if (isEmpty())
{
printf("\nEmpty Queue\n");
}
else
{
printf("\nFront -> %d", front);
printf("\nItems -> ");
for (i = front; i != rear;
 i = (i + 1) % SIZE)
{
printf("%d", items[i]);
}
printf("%d", items[i]);
printf("\nRear -> %d\n", rear);
}

int main()
{
deQueue();
enQueue(1);
enQueue(2);
enQueue(3);
enQueue(4);
```

```
enQueue(5);
enQueue(6);
display();
deQueue();
display();
enQueue(7);
display();
enQueue(8);
return 0;
```

Output:

```
Queue is empty!!

Inserted -> 1
Inserted -> 2
Inserted -> 3
Inserted -> 4
Inserted -> 5
Queue is full!!

Front -> 0
Items -> 1 2 345
Rear -> 4
Deleted element -> 1
Front -> 1
Items -> 2 3 45
Rear -> 4
Inserted -> 7
Front -> 1
Items -> 2 3457
Rear -> 0
Queue is full!!
```